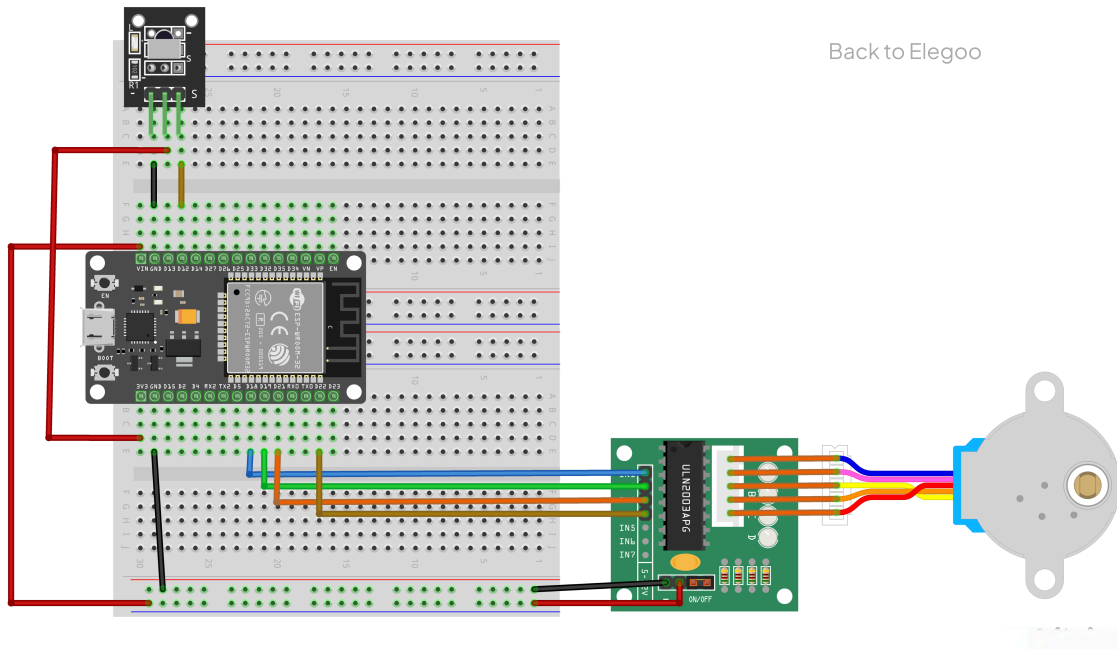




Code

You can click the blue text link to download the program file to your local device, and double-click the file to open it after the download is complete. Please note: Before opening the file, ensure that you have installed the Arduino IDE development environment and completed the installation of relevant components such as the board support package and driver corresponding to the ESP32 development board. If you have any questions about this operation process, you can refer to the "part 1" chapter of the document for detailed guidance.

[DOWENLOAD with_remote.zip](#)

Before you can run this, make sure that you have installed the library or re-install it, if necessary. Otherwise, your code won't work.

With this code, the ESP32 controls a 28BYJ-48 stepper motor using an IR remote control. The motor can be rotated clockwise or counterclockwise, will automatically stop after 3 seconds of inactivity, and enter sleep mode after 5 seconds to save power.

```
#include <Stepper.h>
```

Library Include:

- `#include <Stepper.h>` : Arduino Stepper library for controlling stepper motors

```
// Stepper motor parameters (28BYJ-48 + ULN2003 compatible)
const int stepsPerRevolution = 2048; // Steps per full rotation (standard for 28BYJ-48)
const int motorSpeed = 10; // Rotation speed (5-15 rpm, higher = faster)
```

Stepper Motor Parameters:

- `const int stepsPerRevolution = 2048;` : Steps per full rotation for 28BYJ-48 stepper motor
- `const int motorSpeed = 10;` : Motor rotation speed in RPM (revolutions per minute)

```
// Motor pin definitions (extract individual pins for later sleep control)
const int MOTOR_PIN1 = 5;
const int MOTOR_PIN2 = 19;
const int MOTOR_PIN3 = 18;
const int MOTOR_PIN4 = 21;
Stepper myStepper(stepsPerRevolution, MOTOR_PIN1, MOTOR_PIN2, MOTOR_PIN3, MOTOR_PIN4); // LN1-LN3-LN2-LN4
```

Motor Pin Configuration:

- Individual pin definitions for later sleep control:
 - `const int MOTOR_PIN1 = 5;`
 - `const int MOTOR_PIN2 = 19;`

- `const int MOTOR_PIN3 = 18;`
- `const int MOTOR_PIN4 = 21;`
- `Stepper myStepper(stepsPerRevolution, MOTOR_PIN1, MOTOR_PIN2, MOTOR_PIN3, MOTOR_PIN4);` : Creates Stepper object with specified pins
- Pin mapping: LN1 → GPIO 5, LN3 → GPIO 19, LN2 → GPIO 18, LN4 → GPIO 21

```
// IR receiver pin
const int IR_PIN = 25;
```

IR Receiver Pin:

- `const int IR_PIN = 25;` : Pin connected to IR receiver module

```
// Motor state variables
bool isRunning = false; // Running state flag
bool isSleeping = false; // Sleep state flag (new)
int motorDirection = 1; // 1=clockwise, -1=counterclockwise
unsigned long lastCmdTime = 0; // Last command received time
const unsigned long STOP_DELAY = 3000; // Auto-stop delay with no operation (3 seconds)
const unsigned long SLEEP_DELAY = 5000; // Delay before entering deep sleep (5 seconds, can be same as STOP_DELAY)
```

Motor State Variables:

- `bool isRunning = false;` : Flag indicating if motor is currently running
- `bool isSleeping = false;` : Flag indicating if motor is in sleep mode (new)
- `int motorDirection = 1;` : Motor rotation direction (1=clockwise, -1=counterclockwise)
- `unsigned long lastCmdTime = 0;` : Timestamp of last IR command received
- `const unsigned long STOP_DELAY = 3000;` : Auto-stop delay (3 seconds)
- `const unsigned long SLEEP_DELAY = 5000;` : Delay before entering sleep mode (5 seconds)

```
// Read IR pulse duration (in microseconds)
unsigned long readPulse(int level, unsigned long timeout = 20000){
  unsigned long start = micros();
  while (digitalRead(IR_PIN) == level){
    if (micros() - start > timeout) return 0; // Return 0 on timeout
  }
  return micros() - start;
}
```

Read IR Pulse Function:

- `unsigned long readPulse(int level, unsigned long timeout = 20000)` : Reads the duration of an IR pulse
- `unsigned long start = micros();` : Records start time
- `while (digitalRead(IR_PIN) == level){ ... }` : Waits until pin level changes
- `if (micros() - start > timeout) return 0;` : Returns 0 if timeout occurs
- `return micros() - start;` : Returns pulse duration in microseconds

```
// Motor sleep function (set all motor pins to INPUT high-impedance state) (new)
void motorSleep(){
  if (isSleeping) return; // Avoid repeated execution
  pinMode(MOTOR_PIN1, INPUT);
  pinMode(MOTOR_PIN2, INPUT);
  pinMode(MOTOR_PIN3, INPUT);
  pinMode(MOTOR_PIN4, INPUT);
  isSleeping = true;
  Serial.println("Status: Motor entered sleep mode (low power, no heat)");
}
```

Motor Sleep Function (New):

- `void motorSleep()` : Puts motor into sleep mode to save power

- `if (isSleeping) return;` : Avoids repeated execution
- Sets all motor pins to INPUT mode (high-impedance state) to reduce power consumption and heat
- `isSleeping = true;` : Updates sleep state flag
- Prints status message to serial monitor

[Back to Elegoo](#)

```
// Motor wake-up function (restore motor pins to OUTPUT for operation) (new)
void motorWakeup() {
  if (!isSleeping) return; // Avoid repeated execution
  pinMode(MOTOR_PIN1, OUTPUT);
  pinMode(MOTOR_PIN2, OUTPUT);
  pinMode(MOTOR_PIN3, OUTPUT);
  pinMode(MOTOR_PIN4, OUTPUT);
  myStepper.setSpeed(motorSpeed); // Restore motor speed configuration
  isSleeping = false;
  Serial.println("Status: Motor woken up, ready for operation");
}
```

Motor Wake-up Function (New):

- `void motorWakeup()` : Wakes motor from sleep mode
- `if (!isSleeping) return;` : Avoids repeated execution
- Sets all motor pins back to OUTPUT mode
- `myStepper.setSpeed(motorSpeed);` : Restores motor speed configuration
- `isSleeping = false;` : Updates sleep state flag
- Prints status message to serial monitor

```
// Process IR code and control motor
void handleIRCode(unsigned long code) {
  Serial.print("IR Code: 0x");
  Serial.println(code, HEX); // Print raw code for debugging

  switch (code) {
    case 0xFF629D: // VOL+ code (update with your actual code from serial monitor)
      motorWakeup(); // Wake up motor before operation (new)
      motorDirection = 1; // Clockwise rotation
      isRunning = true;
      lastCmdTime = millis();
      Serial.println("Command: VOL+ → Clockwise");
      break;
    case 0xFFA857: // VOL- code (update with your actual code from serial monitor)
      motorWakeup(); // Wake up motor before operation (new)
      motorDirection = -1; // Counterclockwise rotation
      isRunning = true;
      lastCmdTime = millis();
      Serial.println("Command: VOL- → Counterclockwise");
      break;
    case 0xFFFFFFFF: // Long press repeat code
      if (isRunning) {
        lastCmdTime = millis(); // Extend running time
        Serial.println("Command: Long Press → Continue Running");
      }
      break;
    default:
      Serial.println("Undefined key, ignored");
      break;
  }
}
```

Handle IR Code Function:

- `void handleIRCode(unsigned long code)` : Processes IR remote control commands
- `Serial.print("IR Code: 0x"); Serial.println(code, HEX);` : Prints raw IR code for debugging
- `switch (code) { ... }` : Handles different IR commands
 - `case 0xFF629D:` : VOL+ button → wake up motor and set clockwise rotation
 - `case 0xFFA857:` : VOL- button → wake up motor and set counterclockwise rotation

- **case 0xFFFFFFFF** : Long press → extend running time
- **default** : Undefined key → ignore

[Back to Elegoo](#)

```
void setup(){
  // Initialize motor pins (explicitly set to OUTPUT for initial operation) (new)
  pinMode(MOTOR_PIN1, OUTPUT);
  pinMode(MOTOR_PIN2, OUTPUT);
  pinMode(MOTOR_PIN3, OUTPUT);
  pinMode(MOTOR_PIN4, OUTPUT);

  // Initialize motor
  myStepper.setSpeed(motorSpeed);

  // Initialize IR pin
  pinMode(IR_PIN, INPUT);

  // Initialize serial communication (baud rate 9600)
  Serial.begin(9600);
  Serial.println("=== System Started Successfully ===");
  Serial.println("Operation Guide:");
  Serial.println("VOL+ → Motor Clockwise");
  Serial.println("VOL- → Motor Counterclockwise");
  Serial.println("Auto-stop after 3 seconds of inactivity, auto-sleep after 5 seconds");
  Serial.println("-----");
}
```

Setup Function:

- Explicitly initializes motor pins as OUTPUT (new):
 - **pinMode(MOTOR_PIN1, OUTPUT);**
 - **pinMode(MOTOR_PIN2, OUTPUT);**
 - **pinMode(MOTOR_PIN3, OUTPUT);**
 - **pinMode(MOTOR_PIN4, OUTPUT);**
- **myStepper.setSpeed(motorSpeed);** : Sets motor speed
- **pinMode(IR_PIN, INPUT);** : Sets IR pin as input
- **Serial.begin(9600);** : Initializes serial communication
- Serial prints welcome message and operation guide, including sleep mode information

```
void loop(){
  // IR signal decoding
  if (digitalRead(IR_PIN) == LOW) { // Detect IR start low level
    // Verify leader code (9ms low level)
    unsigned long lowTime = readPulse(LOW);
    if (lowTime < 8000 || lowTime > 10000) return;

    1 // Verify 4.5ms high level after leader code
    2 unsigned long highTime = readPulse(HIGH);
    3 if (highTime < 4000 || highTime > 5000) return;
    4
    5 // Read 32-bit data code
    6 unsigned long code = 0;
    7 for (int i = 0; i < 32; i++) {
    8   // Bit start low level (560us)
    9   unsigned long bitLow = readPulse(LOW);
    10  if (bitLow < 400 || bitLow > 700) return;
    11
    12  // Bit data high level (560us=0, 1680us=1)
    13  unsigned long bitHigh = readPulse(HIGH);
    14  if (bitHigh == 0) return;
    15
    16  code <<= 1;
    17  if (bitHigh > 1000) code |= 1; // High level >1000us is considered 1
    18  }
    19
    20  handleIRCode(code); // Process decoded command
    21  delay(50); // Debounce delay
  }
```

```
}

// Motor operation control
if (isRunning){
  // Check for auto-stop timeout
  if (millis() - lastCmdTime > STOP_DELAY){
    isRunning = false;
    Serial.println("Status: Auto-stopped due to timeout");
  } else {
    // Continuous rotation (1 step at a time to avoid blocking program)
    myStepper.step(motorDirection);
  }
} else {
  // Enter sleep mode if stopped and no operation for SLEEP_DELAY (new)
  if (millis() - lastCmdTime > SLEEP_DELAY && !isSleeping){
    motorSleep();
  }
}
}
```

Loop Function:

► IR signal decoding:

- `if (digitalRead(IR_PIN) == LOW) { ... }` : Detects IR start signal
- Verifies leader code (9ms low level followed by 4.5ms high level)
- Reads 32-bit IR data code
- `handleIRCode(code);` : Processes decoded command
- `delay(50);` : Debounce delay

► Motor operation control:

- `if (isRunning) { ... }` : Controls motor if running
- `if (millis() - lastCmdTime > STOP_DELAY) { ... }` : Auto-stops motor after inactivity
- `myStepper.step(motorDirection);` : Moves motor one step at a time (non-blocking)
- `else { ... }` : Handles stopped state
- `if (millis() - lastCmdTime > SLEEP_DELAY && !isSleeping) { motorSleep(); }` : Enters sleep mode after extended inactivity (new)

Troubleshooting

- **Motor Not Moving:** Check all wiring connections and ensure proper power supply
- **Incorrect Rotation Direction:** Try swapping motor pins in the Stepper constructor
- **IR Remote Not Working:**
 - Check IR receiver connections
 - Open serial monitor to see raw IR codes
 - Update IR code values in `handleIRCode()` function to match your remote
- **Motor Stalling:** Try reducing motor speed (`motorSpeed`) if motor stalls at higher speeds
- **Sleep Mode Issues:** Ensure motor pins are correctly defined and sleep function is being called
- **No Auto-stop/Auto-sleep:** Check `STOP_DELAY` and `SLEEP_DELAY` values